SONALI BANK PLC

Document Archive Intelligent System

Deployment specification

Storage, RAM, database, vector database, bandwidth, history-log retention, and scale-up guidance for local, Railway, AWS, and DigitalOcean.

Version 1.0 · 18 August 2026 · FastAPI + PostgreSQL 16 + Qdrant 1.13 + Redis 7 + S3-compatible object storage

Workload	Documents	Users	App RAM	Postgres	Qdrant RAM	Object store	Egress
Pilot	500	20	4 GB	10 GB	1 GB	50 GB	50–100 GB/mo
Production	5,000	50	8 GB	20 GB	4 GB	200 GB	200–500 GB/mo
Growth	25,000	80	8 GB (API) + 8 GB (worker)	50 GB	16 GB	1 TB	1 TB+/mo

Recommended production starting point: two 4 GB web replicas, an 8 GB ingest worker, 20 GB Postgres, 4 GB Qdrant, persistent Redis, 200 GB S3/R2/Spaces, STORAGE_BACKEND=s3.

1. What this system actually is

The Sonali Bank Archive System is a FastAPI web tier plus separate worker services. Uploads and meeting jobs are written to a Postgres outbox in the same transaction as the record, relayed to Redis Streams, and processed by ingest, meeting, and notify workers (consumer groups, retries, dead-letter stream). The browser app is a React TypeScript PWA under /app with the server-rendered Jinja pages kept as a fallback. Production originals must live in S3-compatible object storage because Railway (and typical PaaS) disks are ephemeral.

Hard constraints from the current codebase:

- PDF and image uploads (JPEG, PNG, TIFF), 50 MB cap; originals are write-once with a SHA-256 per version.
- OCR at 300 DPI (Tesseract eng+ben + Poppler) runs in the ingest worker — this dominates worker RAM.
- Views are watermarked per viewer, cached briefly, and served as byte ranges (PDF_STAMP_CACHE_MB per web replica).
- Qdrant collections are named by embedding model and dimension behind an alias (1536-d for text-embedding-3-small, 1024-d for bge-m3); 512-token chunks / 50 overlap.
- Redis carries the job streams, the search cache, and shared rate-limit counters — size it as durable, not as a cache.
- Web replicas scale freely; scheduled sweeps (reminders, audit sealing) take a Redis lock so only one replica runs each.

2. Sizing assumptions

Input	Assumption
Typical PDF	5 MB (range 1–20 MB, hard cap 50 MB)
Chunks per document	~25 plus 1 summary vector
Vector bytes	1536 × 4 B 6 KB raw (1024-d bge-m3 4 KB); ~12–16 KB with Qdrant HNSW
Concurrent staff	5–15 typical; 30+ on meeting days
Users	20–80 (admin / board_secretary / uploader / board_member)
View bandwidth	Each successful open 2× PDF size on the wire

Used storage (not provisioned SKU) grows mainly as PDF bytes. At 500 / 5,000 / 25,000 documents: PDF objects ~3.3 / 32.5 / 162.5 GB; Qdrant HNSW ~0.18 / 1.8 / 9.1 GB; Postgres ~0.5 / 2 / 8 GB (metadata + 6 years of audit at ~200 events/day). Postgres is not the cost driver.

3. Workload tiers

Metric	Pilot	Production	Growth
Documents / users	500 / 20	5,000 / 50	25,000 / 80
Concurrency	5–15 typical	15–30 on meeting days	30+ with overlapping OCR
App RAM target	4 GB	8 GB	8 GB API + 8 GB worker
Postgres provisioned	10 GB	20 GB	50 GB
Qdrant RAM	1 GB	4 GB	16 GB
Object storage	50 GB	200 GB	1 TB
Monthly egress	50–100 GB	200–500 GB	1 TB+
Audit history	~220k / 3 yr	~440k / 6 yr	hot + yearly archive

4. Environment specifications

Monthly dollar ranges are 2026 planning order-of-magnitude, not quotes. vCPU/RAM figures are reserved capacity, not average usage. Provisioned SKUs include headroom so OCR spikes and HNSW growth do not page.

4.1 Local server / on-prem

Pilot — Laptop or small office box running docker compose

Region: On-prem / LAN · Cost: Hardware only · Replicas: 1 process (uvicorn --reload) · 500 documents, ~20 users, light concurrent upload

> **Local is for development and internal demo**
>
> PDFs land in UPLOAD_DIR, there is no HA, and docker-compose.yml has no CPU/memory limits. Do not treat this as the durable Sonali Bank archive.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
Host machine	Dev workstation	4 cores min	16 GB (8 GB floor)	50 GB SSD + PDF volume	1 Gbps LAN	App 4 GB + Qdrant 2 GB + Postgres 1 GB + OS
App (FastAPI)	venv or Dockerfile	shares host	4 GB reserved	UPLOAD_DIR local	LAN only	OCR + watermark in-process
PostgreSQL 16	compose postgres:16	1 vCPU	1 GB	postgres_data volume	localhost	bcpdb; no backups unless you add them
Qdrant 1.13	compose qdrant/qdrant:v1.13.0	1 vCPU	2 GB	qdrant_data volume	:6333 / :6334	1536-d cosine; summaries + chunks
Redis 7	compose redis:7-alpine	shared	256 MB	appendonly volume	:6379	Job streams + cache; persistence keeps queued jobs across restarts
PDF storage	local UPLOAD_DIR	—	—	~4 GB used / 50 GB disk	LAN	No lifecycle or replication

Production — On-prem server — only if the archive must stay inside the office

Region: On-prem / VPN · Cost: Server + power · Replicas: 1 uvicorn process · 5,000 documents, ~50 users, meeting-day spikes

> **Prefer object storage even on a local server**
>
> A single disk holding PDFs is a single point of failure. Point STORAGE_BACKEND=s3 at MinIO or a NAS S3 gateway if this must stay on-prem.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
Host machine	Tower / 1U server	8 cores	32 GB	500 GB NVMe + backup	1–10 Gbps LAN	Leave 8 GB for OS and OCR spikes
App (FastAPI)	Docker image, port 8080	2 vCPU	8 GB	container + /data/uploads	LAN / reverse proxy	Tesseract eng+ben, Poppler, 50 MB PDF cap
PostgreSQL 16	compose or native	2 vCPU	2 GB	50 GB + nightly dump	localhost	7-day dump to a second disk
Qdrant 1.13	compose	2 vCPU	4 GB	20 GB volume	localhost	~1.8 GB vectors at 5k docs
Redis 7	compose	shared	512 MB	none	localhost	Fail-open cache, 1h TTL
PDF storage	MinIO or NAS	—	—	200 GB usable	LAN	Do not keep production PDFs on the app disk

Growth — Local/on-prem at 25k docs — split the worker or move to cloud

Region: On-prem only if policy requires · Cost: Server fleet · Replicas: API + worker (do not run 2 schedulers) · 25,000 documents, ~80 users, concurrent OCR + heavy view

Do not scale compose replicas as-is

In-process APScheduler would double meeting reminders, and the login rate limit is in-memory. Extract OCR/chunk indexing to a worker and keep one scheduler.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
API host	App server	4 cores	8 GB	100 GB OS	10 Gbps LAN	Thin API: search, view, meetings
Ingest worker	Same image, different command	4 cores	8 GB	scratch for OCR	LAN to MinIO	Parse, 300 DPI OCR, embed, Qdrant upsert
PostgreSQL 16	dedicated VM	4 vCPU	8 GB	100 GB + backups	private LAN	Partition audit_logs older than 3 years
Qdrant	dedicated VM	4 vCPU	16 GB	80 GB SSD	private LAN	~9 GB HNSW at 25k docs; snapshot weekly
Redis 7	compose or native	shared	1 GB	none	private LAN	Cache + future rate-limit / lock keys
PDF storage	MinIO erasure-coded	—	—	1 TB usable	LAN / 10 Gbps	Versioning off; lifecycle optional

4.2 Railway

Pilot — Railway app + Neon + Qdrant Cloud + Upstash + R2/S3

Region: Railway US/EU + nearest data plane · Cost: ~$40–90 / month · Replicas: 1 Railway service · 500 documents, ~20 users, light concurrent upload

Railway disk is ephemeral

Set STORAGE_BACKEND=s3 (AWS S3 or Cloudflare R2). Local UPLOAD_DIR is lost on every deploy. Health check path is /healthz.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
App	Railway Docker (this Dockerfile)	2 vCPU	4 GB	ephemeral — not for PDFs	shared ~100+ Mbps	PORT from Railway; OCR needs the 4 GB
PostgreSQL	Neon 0.25 CU	shared CU	shared	10 GB	pooled ssl=require	App rewrites postgresql:// to asyncpg
Qdrant	Qdrant Cloud 1 GB	managed	1 GB	managed	TLS :443	QDRANT_URL + QDRANT_API_KEY
Redis	Upstash 256 MB	managed	256 MB	none	rediss://	Or UPSTASH_REDIS_REST_* pair
PDF objects	R2 or S3 Standard	—	—	50 GB provisioned	egress billed	~3 GB used at 500 docs
LLM / embed	OpenRouter or OpenAI	—	—	—	API calls	text-embedding-3-small, 1536-d; switchable per task in /admin/ai

Production — Five services from .railway/railway.ts (web, outbox-relay, three workers)

Region: Railway + ap-south-1 / R2 for objects if serving BD · Cost: ~$180–350 / month · Replicas: web 2× + relay 1× + worker-ingest 1–3× + worker-meeting 1× + worker-notify 1× · 5,000 documents, ~50 users, meeting-day spikes

Scale workers on stream lag, web on HTTP load

Jobs flow from the Postgres outbox to Redis Streams to worker consumer groups. scripts/autoscale_workers.py reads stream lag and sets worker replicas. Scheduled sweeps take a Redis lock, so extra web replicas do not double reminders.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
web	Railway Pro 4 GB × 2	2 vCPU	4 GB	ephemeral	shared; HTTP + byte-range PDF views	Search, view, meetings, Ask; no OCR in the web tier
worker-ingest	Railway Pro 8 GB × 1–3	2 vCPU	8 GB	ephemeral scratch	to S3 + Qdrant + AI	OCR 300 DPI, embed, Qdrant upsert
worker-meeting / notify / relay	Railway 2–4 GB	1–2 vCPU	2–4 GB	ephemeral	low	STT via provider; notify is light; relay claims outbox rows with SKIP LOCKED
PostgreSQL	Neon 1 CU	1 CU	managed	20 GB	pooled	Point-in-time recovery on; nightly encrypted dump via scripts/dr_backup.py
Qdrant	Qdrant Cloud 4 GB	managed	4 GB	managed	TLS	~1.8 GB vectors at 5k docs; headroom for HNSW
Redis	Upstash or Railway Redis 512 MB–1 GB	managed	1 GB	persistence on	rediss://	Job streams + dead letters + cache + rate limits; not ephemeral
PDF objects	S3 / R2	—	—	200 GB	200–500 GB/mo egress	Each view 2× PDF size (fetch + watermarked body)
Email	Resend	—	—	—	low	Invites + 48h/24h reminders

Growth — Split ingest off the web dyno; grow Qdrant and object store

Region: Same as production · Cost: ~$400–800 / month · Replicas: web 2–4× + worker-ingest 2–4× (autoscaled) + meeting/notify/relay · 25,000 documents, ~80 users, concurrent OCR + heavy view

Scale storage and Qdrant first

Postgres metadata stays modest even at 25k docs. PDF bytes and HNSW RAM dominate cost. Raise worker-ingest max replicas for backfills; web replicas follow concurrent users.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
web	Railway 4 GB × 2–4	2 vCPU	4 GB	ephemeral	1 TB+ egress possible	Search, view, meetings, Ask
worker-ingest	Railway 8 GB × 2–4	2 vCPU	8 GB	ephemeral scratch	to S3 + Qdrant + AI	Autoscaled on bcp:stream:ingest backlog
PostgreSQL	Neon 2 CU	2 CU	managed	50 GB	pooled	Yearly archive of audit_logs to R2
Qdrant	Qdrant Cloud 8–16 GB	managed	16 GB	managed snapshots	TLS	~9 GB at 25k docs; consider quantization later
Redis	Upstash or Railway Redis 1–2 GB	managed	2 GB	persistence on	rediss://	Streams, locks, rate limits, cache
PDF objects	S3 / R2 + lifecycle	—	—	1 TB	1 TB+/mo	Do not CDN raw docs — watermark is per-user, Cache-Control: no-store

4.3 Amazon Web Services

Pilot — ECS/Fargate or a single t3.medium + managed data plane

Region: ap-south-1 (Mumbai) or ap-southeast-1 · Cost: ~$80–160 / month · Replicas: 1 task · 500 documents, ~20 users, light concurrent upload

Keep data in-region for Sonali Bank latency

Run the app, RDS, ElastiCache, and S3 in ap-south-1. Qdrant Cloud has no Mumbai region — pick the nearest Qdrant site or run Qdrant on a t3.medium in the same VPC.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
App	ECS Fargate or t3.medium	2 vCPU	4 GB	20 GB EBS (OS only)	Up to 5 Gbps burst	ALB container :8080, health /healthz
PostgreSQL	RDS db.t4g.small	2 vCPU	2 GB	20 GB gp3	VPC	Single-AZ ok for pilot; ssl required
Qdrant	Qdrant Cloud 2 GB or t3.medium	2 vCPU	2 GB	30 GB gp3	VPC / TLS	1536-d cosine collections
Redis	ElastiCache cache.t4g.micro	2 vCPU	0.5 GB	none	VPC	Cache only
PDF objects	S3 Standard	—	—	50 GB	50–100 GB/mo egress	Block public access; app uses get_object then watermarks
Logs	CloudWatch Logs	—	—	90 days hot	—	Operational logs 3–6 months

Production — t3.large API + RDS + S3 + Qdrant 4–8 GB

Region: ap-south-1 · Cost: ~$200–450 / month · Replicas: 1 task / instance · 5,000 documents, ~50 users, meeting-day spikes

Watermarked views double egress

Every /view and /download fetches the object then returns a full stamped PDF. Budget 2× PDF size per request. CloudFront in front of raw S3 is the wrong pattern — docs are no-store and user-specific.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
App	t3.large or Fargate 8 GB	2 vCPU	8 GB	30 GB EBS	Up to 5 Gbps	Meeting-day headroom for concurrent watermarks
PostgreSQL	RDS db.t4g.medium	2 vCPU	4 GB	50 GB gp3	VPC	7-day backup; Multi-AZ if board SLA requires it
Qdrant	Qdrant Cloud 4–8 GB or t3.large	2 vCPU	8 GB	60 GB gp3	VPC / TLS	Snapshot to S3 weekly
Redis	cache.t4g.small	2 vCPU	1.4 GB	none	VPC	Still oversized vs cache need — smallest HA option
PDF objects	S3 + Intelligent-Tiering	—	—	200 GB	200–500 GB/mo	Server access logging on for 12 months
Logs	CloudWatch + S3 export	—	—	90–180 days hot	—	Audit stays in RDS 3–6 years

Growth — Split API and worker; RDS larger; Qdrant 16 GB

Region: ap-south-1 · Cost: ~$600–1,200 / month · Replicas: API 1× + worker 1× · 25,000 documents, ~80 users, concurrent OCR + heavy view

Horizontal scale needs Redis locks first

Before a second API task: move reminder sweep to EventBridge/cron with a Redis lock, and store login rate-limit counters in ElastiCache. Until then, only scale vertically.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
API	Fargate 2 vCPU / 4 GB	2 vCPU	4 GB	20 GB	ALB	No OCR on this task
Ingest worker	Fargate / t3.large 8 GB	2 vCPU	8 GB	scratch	to S3 + Qdrant	OCR 300 DPI is RAM-heavy
PostgreSQL	RDS db.t4g.large	2 vCPU	8 GB	100 GB gp3	VPC	Partition audit_logs; Multi-AZ
Qdrant	r6g.large or Qdrant 16 GB	2 vCPU	16 GB	120 GB + snapshots	VPC	Quantize vectors if RAM > 70%
Redis	cache.t4g.small	2 vCPU	1.4 GB	none	VPC	Cache + rate limit + scheduler lock
PDF objects	S3 Standard / Glacier IR mix	—	—	1 TB	1 TB+/mo	CloudFront only if you cache watermarked bytes per user — usually not worth it

4.4 DigitalOcean

Pilot — App Platform 4 GB + managed Postgres/Redis + Spaces + Qdrant Cloud

Region: BLR1 (Bangalore) or SGP1 · Cost: ~$50–100 / month · Replicas: 1 App Platform instance · 500 documents, ~20 users, light concurrent upload

Spaces speaks the S3 API this app already uses

Set AWS_S3_ENDPOINT_URL to the Spaces endpoint, AWS_BUCKET_NAME to the Space, and STORAGE_BACKEND=s3. A single 8 GB droplet running compose is fine only behind VPN.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
App	App Platform 4 GB (2 GB is demo-only)	2 vCPU	4 GB	container ephemeral	included tier + overage	2 GB will OOM on 300 DPI OCR
PostgreSQL	Managed PG 1 vCPU	1 vCPU	1–2 GB	10 GB	VPC / public TLS	Daily backups included
Qdrant	Qdrant Cloud 1 GB or 4 GB droplet	2 vCPU	1–4 GB	25 GB volume	private net	Prefer Cloud over stuffing it on the app droplet
Redis	Managed Redis 1 GB	shared	1 GB	none	VPC	Smallest managed size is already enough
PDF objects	Spaces 250 GB	—	—	250 GB	1 TB included typical	~3 GB used at 500 docs
Alt all-in-one	Droplet 8 GB / 4 vCPU	4 vCPU	8 GB	160 GB	2–4 Gbps	Compose stack; VPN only — not public prod

Production — 8 GB App Platform or dedicated droplet + managed data + Spaces

Region: BLR1 / SGP1 · Cost: ~$140–280 / month · Replicas: 1 instance · 5,000 documents, ~50 users, meeting-day spikes

Dedicated 8 GB for the app, not a shared 2 GB

Watermarking and OCR share the same process. Meeting-day concurrent views need the extra RAM more than extra instances.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
App	App Platform / droplet 8 GB	4 vCPU	8 GB	OS only	4–5 TB included typical	Health check /healthz
PostgreSQL	Managed PG 2 vCPU	2 vCPU	4 GB	30 GB	VPC	Standby optional
Qdrant	8 GB droplet + volume	4 vCPU	8 GB	80 GB volume	private net	Or Qdrant Cloud 4–8 GB
Redis	Managed Redis 1 GB	shared	1 GB	none	VPC	Cache only
PDF objects	Spaces 1 TB	—	—	1 TB	200–500 GB/mo extra possible	CDN in front of Spaces does not help watermarked no-store PDFs
Email	Resend	—	—	—	low	Same as other clouds

Growth — API droplet + worker droplet + managed PG 4 vCPU + 16 GB Qdrant

Region: BLR1 / SGP1 · Cost: ~$400–750 / month · Replicas: API 1× + worker 1× · 25,000 documents, ~80 users, concurrent OCR + heavy view

Grow Spaces and Qdrant; Postgres stays modest

At 25k docs the object store is ~160 GB used (plan 1 TB). Qdrant wants 16 GB RAM. Postgres 60 GB is still mostly empty if you archive audit rows yearly.

Service	SKU	vCPU	RAM	Disk	Bandwidth	Notes
API	Droplet 4 GB / 2 vCPU	2 vCPU	4 GB	80 GB	4–5 TB included	No OCR
Ingest worker	Droplet 8 GB / 4 vCPU	4 vCPU	8 GB	80 GB scratch	private + Spaces	OCR + embeddings
PostgreSQL	Managed PG 4 vCPU	4 vCPU	8 GB	60 GB	VPC	Archive audit_logs > 3 years to Spaces
Qdrant	Droplet 16 GB + volume	8 vCPU	16 GB	200 GB volume	private net	Snapshots to Spaces
Redis	Managed Redis 1 GB	shared	1 GB	none	VPC	Add lock keys before a second API droplet
PDF objects	Spaces 1 TB+	—	—	1 TB+	1 TB+/mo possible	Spaces CDN optional for static PWA assets only

5. History logs (3–6 months and 3–6 years)

Six streams. Keep board accountability in Postgres for years; drop noisy operational logs after a quarter or two. There is no TTL in code today — add a yearly archive job (CSV/Parquet to object storage) rather than deleting audit rows.

Stream	Store	What is recorded	Retain	Volume
Audit trail	Postgres audit_logs	login, upload, view, download, access_request, user admin	3–6 years hot	~0.5–2 KB/row; <2 GB at 6 years
App / uvicorn	Railway logs, CloudWatch, journald	OCR failures, index errors, 5xx	3–6 months	tens of MB/month
Access / proxy	ALB, Railway edge, Nginx	HTTP method, status, latency	3–6 months	depends on QPS; drop after debug window
Meeting notifications	Postgres notification_events	invites, 48h/24h reminders	3–6 years (with meetings)	small; meetings × invitees
Object access	S3 / R2 / Spaces server logs	who fetched which key	12 months	low; enable on the bucket
LLM / embedding usage	OpenAI or OpenRouter dashboard	tokens, spend, model	12 months of invoices	external — not in this DB

At 200 audited actions/day: ~73k rows/year, ~220k at 3 years, ~440k at 6 years — well under 2 GB. Do not scale Postgres for logs. Scale it for backup RPO and connection pool. PDF bytes dominate cost.

Operational logs (app, proxy, meeting debug): 3–6 months hot. Audit trail and meeting notification events: 3–6 years. Object-storage access logs: 12 months. LLM usage: 12 months of invoices in the vendor dashboard.

6. How to scale when traffic grows

Order of operations: object storage Qdrant RAM app RAM (4 8 GB) ingest worker replicas (only after Redis locks). Do not horizontally scale the current Dockerfile: a second web process would duplicate meeting reminder emails and split the in-memory login rate limit.

Trigger	Next step
Archive > ~500 docs, or Railway/DO disk used for PDFs	Move to S3 / R2 / Spaces immediately (STORAGE_BACKEND=s3)
Qdrant RAM > 70% or search latency climbs	Next Qdrant RAM tier (HNSW is RAM-bound). 1 4 8 16 GB
Postgres disk > 70%	Grow volume; partition/archive audit_logs older than 3 years
Concurrent OCR uploads stall the UI	Extract parse/OCR + _index_document_chunks to a worker; keep API thin
Meeting-day view storms / 502s	Add web replicas (scheduled sweeps are lock-protected); raise PDF_STAMP_CACHE_MB
Egress cost spikes	Accept 2× PDF bytes per view. Do not CDN raw docs (watermark + no-store)
Need 2+ app replicas	Externalize reminders (cron + Redis lock) and login rate limit (Redis) first
25k+ documents	Qdrant 16 GB; consider quantized vectors. Postgres is still modest

Scale blockers in this codebase

• Single replica — APScheduler reminder sweep and in-memory login rate limit (8 failures / 10 min) are process-local.

• In-process OCR — parse_pdf + Tesseract 300 DPI + LLM summary run on the same uvicorn process as search and meetings.

• Watermark bandwidth — view and download pull the full object, stamp it in memory, and return the whole PDF with Cache-Control: no-store.

7. Production checklist

• APP_ENV=production, JWT_SECRET_KEY 32 characters, COOKIE_SECURE on.

• DATABASE_URL pointing at managed Postgres (Neon / RDS / DO Managed PG). App rewrites postgresql:// to asyncpg.

• QDRANT_URL + QDRANT_API_KEY (or self-hosted Qdrant with persistent volume).

• REDIS_URL=rediss://… or Upstash REST pair. Do not leave redis://127.0.0.1 on a PaaS.

• STORAGE_BACKEND=s3 with AWS_BUCKET_NAME, keys, region; set AWS_S3_ENDPOINT_URL for R2/Spaces/MinIO.

• OPENAI_API_KEY or OPENROUTER_API_KEY for summaries and text-embedding-3-small.

• Load balancer checks /readyz. Two or more web replicas; workers scale on stream backlog.

• Operational log retention 3–6 months; audit_logs retained 3–6 years with a yearly archive job.

Sources: Dockerfile, docker-compose.yml, railway.toml, src/bcp_project/main_api.py, models.py, qdrant_store.py, chunker.py. Monthly costs are planning estimates only.